Logging, Actuator, Prometheus & Grafana (Days 62-65)

1. Logback config (Day 62)

Create `src/main/resources/logback-spring.xml`:

```
<configuration>
    <include resource="org/springframework/boot/logging/logback/defaults.xml"/>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} %-5level [%thread] %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <!-- Your own code: INFO. Framework internals: WARN, to cut noise. -->
    <logger name="org.example.enterprisedigitalbankingsystem" level="INFO"/>
    <logger name="org.hibernate.SQL" level="DEBUG"/>
    <logger name="org.springframework" level="WARN"/>

    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>
</configuration>
```

Already-used `@Slf4j` (Lombok) classes — `TransactionEventProducer`, `BeneficiaryEventProducer` — will now respect these levels automatically.

2. Spring Boot Actuator (Day 63)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=always
```

Test: `curl http://localhost:8080/actuator/health` → `{"status":"UP"}` (also shows DB connection health). `curl .../actuator/metrics` lists every available metric name; `curl .../actuator/metrics/jvm.memory.used` gives one specific value.

3. Prometheus (Day 64)

```xml
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

That's it — `/actuator/prometheus` now exists automatically (Micrometer + the exposure list above). Test: `curl http://localhost:8080/actuator/prometheus` — plain-text metrics in Prometheus's scrape format.

Run Prometheus itself to actually scrape it:

```yaml
# prometheus.yml
scrape_configs:
  - job_name: 'banking-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['host.docker.internal:8080']
```

```
docker run -d --name prometheus -p 9090:9090 \
  -v "$(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml" prom/prometheus
```

Test: open `http://localhost:9090`, go to Status → Targets, confirm `banking-app` shows `UP`.

4. Grafana (Day 65)

```
docker run -d --name grafana -p 3000:3000 grafana/grafana
```

Open `http://localhost:3000` (default login `admin` / `admin`), add a data source → Prometheus → URL `http://host.docker.internal:9090` → Save & Test. Import dashboard ID `4701` ("JVM Micrometer") from grafana.com/dashboards for an instant working dashboard against your app's JVM metrics (heap, threads, GC) — confirms the whole pipeline (app → Prometheus → Grafana) without having to hand-build panels.

Test: watch the dashboard update live while you hit a few banking app endpoints with curl — request counts / heap usage should visibly move.